

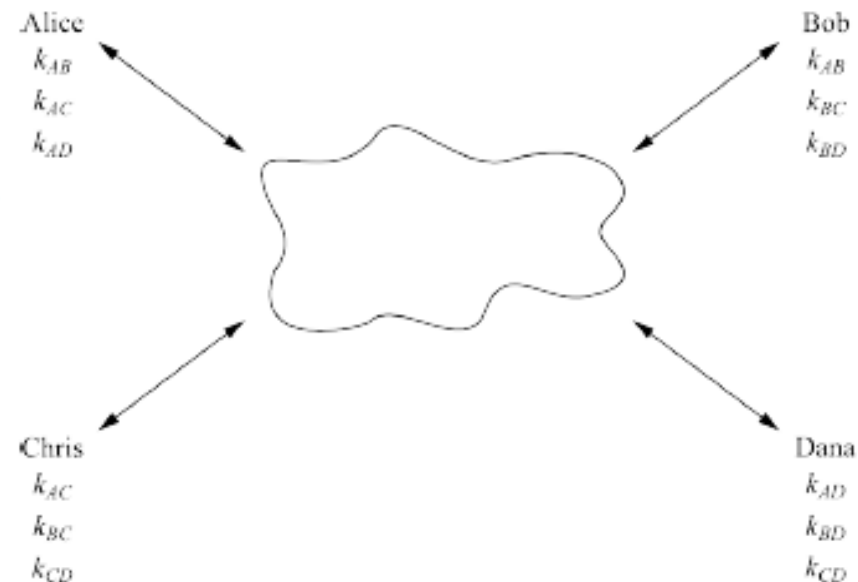
Cryptographie
Chapitre
Introduction à la cryptographie à clé
publique

Échange de secret

■ The n^2 Key Distribution Problem

Shortcomings

- There are $n(n-1) \approx n^2$ keys in the system
 - There are $n(n-1)/2$ key pairs
 - If a new user Esther joins the network, new keys k_{XE} have to be transported via secure channels (!) to each of the existing users
- ⇒ Only works for small networks which are relatively static



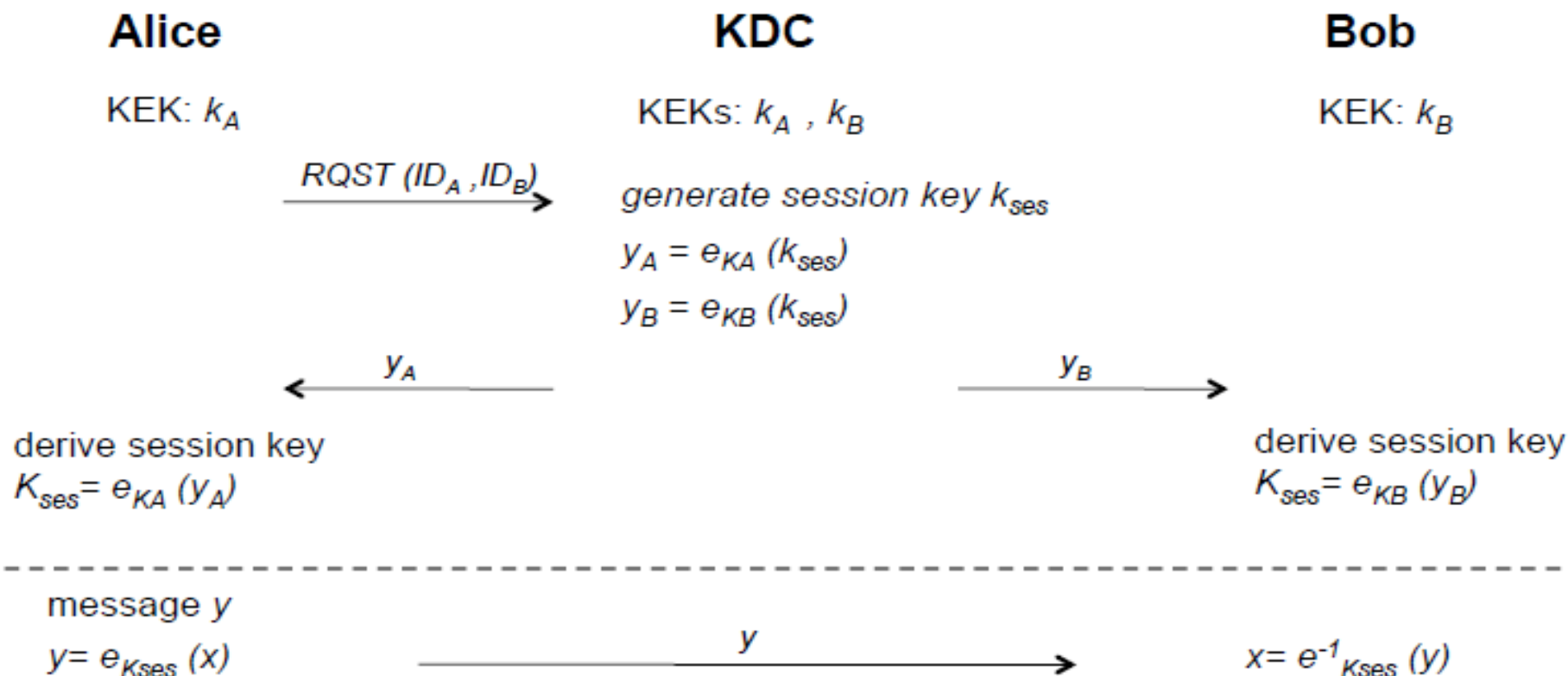
Example: mid-size company with 750 employees

- $750 \times 749 = 561,750$ keys must be distributed securely

Échange de secret

■ Key Establishment with Key Distribution Center

- Key Distribution Center (KDC) = Central party, trusted by all users
- KDC shares a key encryption key (KEK) with each user
- Principle: KDC sends session keys to users which are encrypted with KEKs



Échange de secret

■ Key Establishment with Key Distribution Center

Remaining problems:

- **No *Perfect Forward Secrecy***: If the KEKs are compromised, an attacker can decrypt past messages if he stored the corresponding ciphertext
- **Single point of failure**: The KDC stores all KEKs. If an attacker gets access to this database, all past traffic can be decrypted.
- **Communication bottleneck**: The KDC is involved in every communication in the entire network (can be countered by giving the session keys a long life time)
- For more advanced attacks (e.g., key confirmation attack): Cf. Section 13.2 of *Understanding Cryptography*

Échange de secret

■ Diffie–Hellman Key Exchange (DHKE)

Alice

Public parameters α, p

Bob

Choose random private key
 $k_{prA} = a \in \{1, 2, \dots, p-1\}$

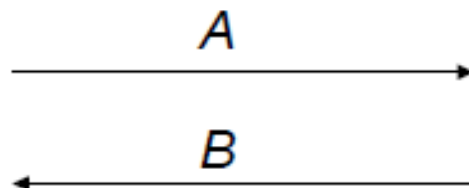
Compute public key
 $k_{pubA} = A = \alpha^a \bmod p$

Compute common secret
 $k_{AB} = B^a = (\alpha^a)^b \bmod p$

Choose random private key
 $k_{prB} = b \in \{1, 2, \dots, p-1\}$

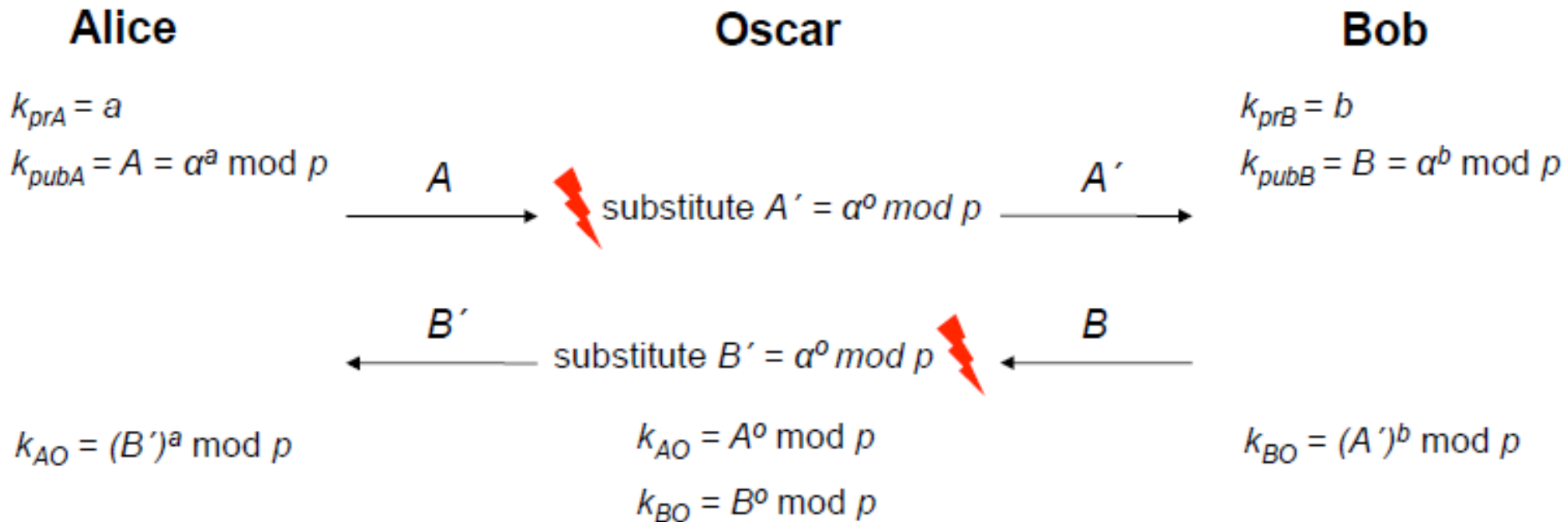
Compute public key
 $k_{pubB} = B = \alpha^b \bmod p$

Compute common secret
 $k_{AB} = A^b = (\alpha^b)^a \bmod p$



- Widely used in practice
- If the parameters are chosen carefully (especially a prime $p > 2^{1024}$), the DHKE is secure against *passive* (i.e., listen-only) attacks
- However: If the attacker can *actively* intervene in the communication, the **man-in-the-middle attack** becomes possible

■ Man-in-the-Middle Attack



- Oscar computes a session key k_{AO} with Alice, and k_{BO} with Bob
- However, Alice and Bob think they are communicating with each other !
- The attack efficiently performs 2 DH key-exchanges: Oscar-Alice and Oscar-Bob
- Here is why the attack works:

Alice computes: $k_{AO} = (B')^a = (\alpha^0)^a$

Oscar computes: $k_{A^0} = A^0 = (\alpha^a)^0$

Bob computes: $k_{BO} = (A')^b = (\alpha^a)^b$

Oscar computes: $k_{B0} = B^0 = (\alpha^a)^0$

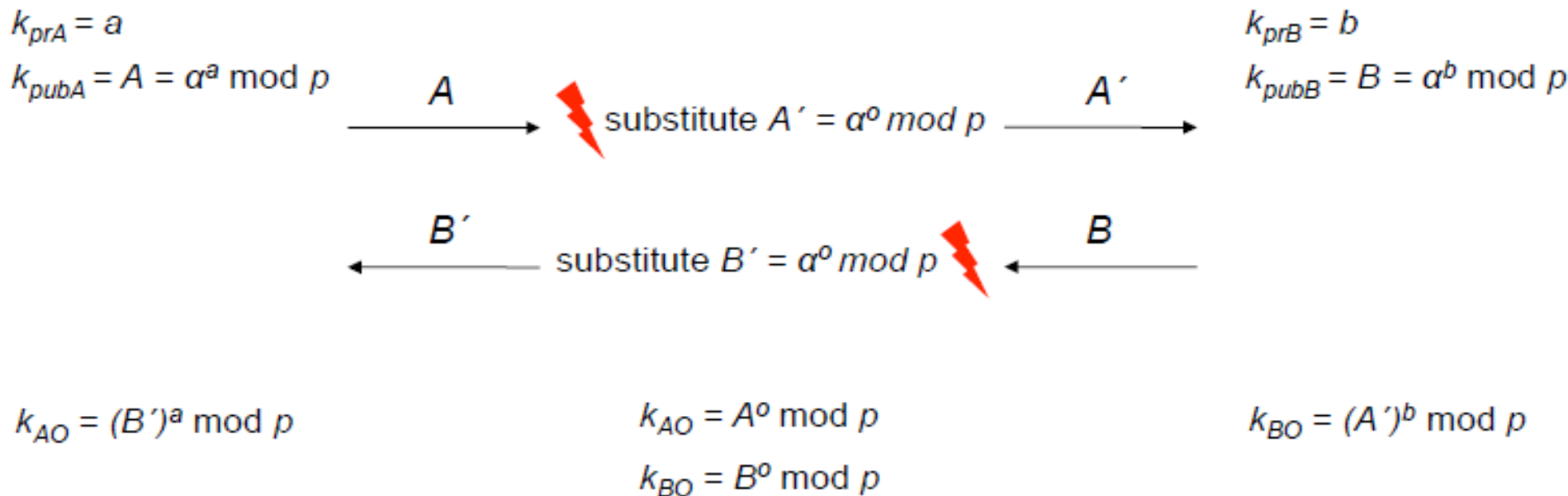
Échange de secret

■ Implications of the Man-in-the-Middle Attack

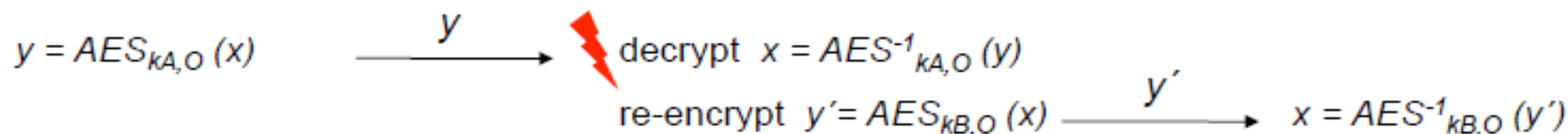
Alice

Oscar

Bob



- Oscar has now complete control over the channel, e.g., if Alice wants to send an encrypted message x to Bob, Oscar can read the message:



Échange de secret

■ Very, very important facts about the Man-in-the-Middle Attack

- The man-in-the-middle-attack is not restricted to DHKE; it is applicable to any public-key scheme, e.g. RSA encryption. ECDSA digital signature, etc. etc.
- The attack works always by the same pattern: Oscar replaces the public key from one of the parties by his own key.
- The attack is also known as MIM attack or Janus attack
- Q: What is the underlying problem that makes the MIM attack possible?
- A: The public keys are not authenticated: When Alice receives a public key which is allegedly from Bob, she has no way of knowing whether it is in fact his. (After all, a key consists of innocent bits; it does not smell like Bob's perfume or anything like that)



Even though public keys can be sent over unsecure channels, they
require authenticated channels.

Échange de secret

■ Certificates

- In order to authenticate public keys (and thus, prevent the MIM attack) , all public keys are digitally signed by a central trusted authority.

- Such a construction is called *certificate*

certificate = public key + ID(user) + digital signature over public key and ID

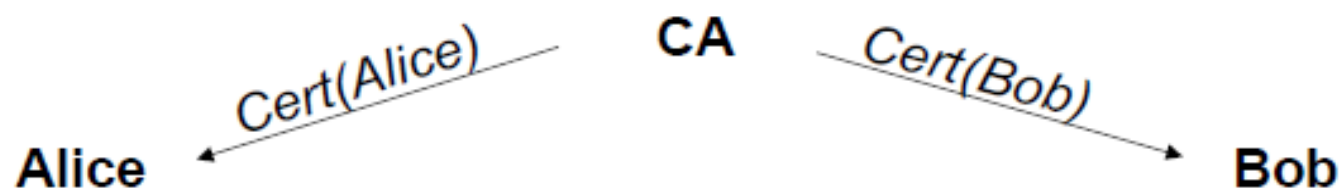
- In its most basic form, a certificate for the key k_{pub} of user Alice is:

$$\mathbf{Cert(Alice) = (k_{pub}, ID(Alice), sig_{K_{CA}}(k_{pub}, ID(Alice)))}$$

- Certificates bind the identity of user to her public key
- The trusted authority that issues the certificate is referred to as **certifying authority (CA)**
- „Issuing certificates“ means in particular that the CA computes the signature $sig_{K_{CA}}(k_{pub})$ using its (super secret!) private key k_{CA}
- The party who receives a certificate, e.g., Bob, verifies Alice's public key using the public key of the CA

Échange de secret

■ Diffie–Hellman Key Exchange (DHKE) with Certificates



$$k_{prA} = a$$

$$k_{pubA} = A$$

$$Cert(Alice) = ((A, ID_A), sig_{K_{CA}}(A, ID_A))$$

$$k_{prB} = b$$

$$k_{pubB} = B = \alpha^b \bmod p$$

$$Cert(Bob) = ((B, ID_B), sig_{K_{CA}}(B, ID_B))$$

$Cert(Alice)$

$Cert(Bob)$

verify certificate

$$ver_{K_{pub, CA}}(Cert(Bob))$$

if verification is correct:

Compute common secret

$$k_{AB} = B^a = (\alpha^a)^b \bmod p$$

verify certificate

$$ver_{K_{pub, CA}}(Cert(Alice))$$

if verification is correct:

Compute common secret

$$k_{AB} = A^b = (\alpha^b)^a \bmod p$$

Échange de secret

■ Certificates

- Note that verification requires the public key of the CA for $ver_{k_{pub}, CA}$
- In principle, an attacker could run a MIM attack when $k_{pub, CA}$ is being distributed
⇒ The public CA keys must also be distributed via an authenticated channel!
- Q: So, have we gained anything?
After all, we try to protect a public key (e.g., a DH key) by using yet another public-key scheme (digital signature for the certificate)?
- A: YES! The difference from before (e.g., DHKE without certificates) is that **we only need to distribute the public CA key *once***, often at the set-up time of the system
- Example: Most web browsers are shipped with the public keys of many CAs. The „authenticated channel“ is formed by the (hopefully) correct distribution of the original browser software.

- Une vue d'ensemble
- Le système RSA, du nom de ses concepteurs Rivest, Shamir et Adleman est le premier système de chiffrement à clé publique robuste à avoir été inventé (en 1977).
- Il est toujours largement utilisé lorsque l'on veut échanger des données de manière sécurisée sur Internet.
- Le chiffrement RSA est asymétrique : il utilise une paire de clés, une clé publique pour le chiffrement et une clé privée pour le déchiffrement.
- Le système RSA permet également de signer des données. Celles-ci sont signées avec la clé privée et tout possesseur de la clé publique peut vérifier la signature.
- La robustesse du système RSA repose sur le fait que l'on ne sait pas, avec les moyens et savoirs actuels, obtenir la clé privée à partir de la simple connaissance de la clé publique.

RSA

- Comme tout système à clé publique, le chiffrement RSA est coûteux en temps calcul. Plus précisément, il est (beaucoup) plus coûteux qu'un chiffrement symétrique de robustesse équivalente comme AES par exemple.
- C'est pour cela, que souvent en pratique, on ne l'utilise que pour transmettre la clé secrète d'un chiffrement symétrique.
- Concrètement supposons qu'Alice veuille échanger des données de manière sécurisée avec Bob. Elle va ainsi procéder :
 - Elle va tout d'abord générer une clé secrète pour le chiffrement symétrique dont le choix (public) a été fait au préalable avec son correspondant.
 - Elle chiffre cette clé secrète avec la clé publique (RSA) de Bob et transmet la clé ainsi chiffrée à ce dernier.
 - Bob déchiffre la clé secrète avec sa clé privée (RSA).
 - Les deux correspondants étant maintenant en possession de la clé secrète, le chiffrement symétrique des données peut commencer.

RSA

- Comme tout système à clé publique, le chiffrement RSA est coûteux en temps calcul. Plus précisément, il est (beaucoup) plus coûteux qu'un chiffrement symétrique de robustesse équivalente comme AES par exemple.
- C'est pour cela, que souvent en pratique, on ne l'utilise que pour transmettre la clé secrète d'un chiffrement symétrique.
- Concrètement supposons qu'Alice veuille échanger des données de manière sécurisée avec Bob. Elle va ainsi procéder :
 - Elle va tout d'abord générer une clé secrète pour le chiffrement symétrique dont le choix (public) a été fait au préalable avec son correspondant.
 - Elle chiffre cette clé secrète avec la clé publique (RSA) de Bob et transmet la clé ainsi chiffrée à ce dernier.
 - Bob déchiffre la clé secrète avec sa clé privée (RSA).
 - Les deux correspondants étant maintenant en possession de la clé secrète, le chiffrement symétrique des données peut commencer.

RSA

Indicatrice d'Euler

Avant d'aborder l'architecture de RSA, nous aurons encore besoin d'un outil mathématique : l'indicatrice d'Euler.

Définition (indicatrice d'Euler)

L'indicatrice d'Euler est la fonction $\varphi : \mathbb{N}^* \rightarrow \mathbb{N}^*$ définie par :

$$\varphi(n) = \text{card}(\{m \in \mathbb{N}^* \mid m \leq n \text{ et } \text{pgcd}(m, n) = 1\}).$$

Par exemple, on a : $\varphi(1) = 1$, $\varphi(2) = 1$, $\varphi(10) = 4$ et, pour tout $p \in \mathbb{P}$, $\varphi(p) = p - 1$.

Proposition

L'indicatrice d'Euler est une fonction multiplicative : si $m, n \in \mathbb{N}^*$ sont premiers entre eux, alors $\varphi(mn) = \varphi(m)\varphi(n)$.

Par conséquent si $n = p_1^{\alpha_1} \cdots p_k^{\alpha_k}$, on a $\varphi(n) = n(1 - \frac{1}{p_1}) \cdots (1 - \frac{1}{p_k})$.

RSA

Indicatrice d'*Euler*

Une généralisation du petit théorème de *Fermat* due à *Euler* :

Théorème (*Euler-Fermat*)

Soient $a, n \in \mathbb{N}^*$ tels que a et n soient premiers entre eux. Alors on a :

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Une application amusante : quel est le chiffre des unités de 2017^{2016} ?

Comme $\varphi(10) = 4$ et $\text{pgcd}(10, 2017) = 1$, on a :

$$2017^{2016} = 2017^{4 \times 504} = (2017^4)^{504} = (2017^{\varphi(10)})^{504} \equiv 1 \pmod{10},$$

le chiffre des unités de 2017^{2016} est donc 1.

Une identité remarquable (toujours due à *Euler*) :

$$\sum_{d|n} \varphi(d) = n$$

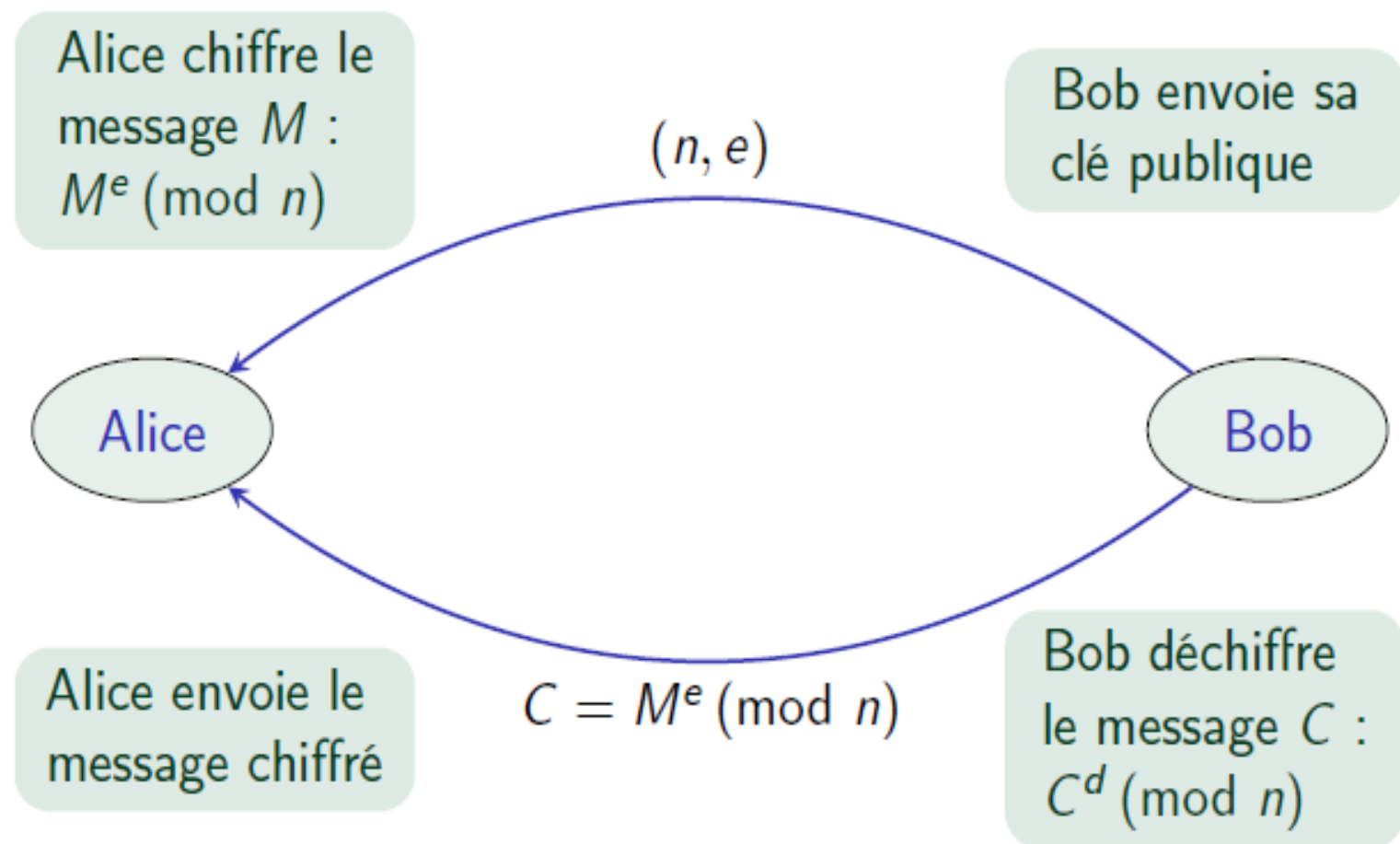
RSA

Le principe

- La **clé privée** est constituée de deux “grands” nombres premiers p et q .
- La **clé publique** est constituée du produit $n = pq$ et d'un entier e inversible modulo $\varphi(n)$, c'est à dire un entier e tel qu'il existe un entier d avec $ed \equiv 1 \pmod{\varphi(n)}$.
- Le **chiffré d'un message** représenté par un entier M modulo n est $M^e \pmod{n}$.
- Pour **déchiffrer le message** chiffré C , il suffit de calculer $C^d \pmod{n}$ où d est l'inverse de e modulo $\varphi(n)$.
- Justification de l'algorithme :
 - Supposons que M soit premier avec n . Alors, d'après le théorème d'*Euler-Fermat*, on a : $M^{\varphi(n)} \equiv 1 \pmod{n}$. Par conséquent :
$$C^d = (M^e)^d = M^{ed} = M^{1+k\varphi(n)} = M \times (M^{\varphi(n)})^k \equiv M \pmod{n}$$
 - Lorsque M n'est pas premier avec n , trois cas se présentent : $p \mid M$ et $q \nmid M$; $p \nmid M$ et $q \mid M$; $n \mid M$. Le dernier cas est trivial tandis que les deux premiers nécessitent l'utilisation du *théorème chinois* en plus du théorème d'*Euler-Fermat* pour conclure.

RSA

Schéma de fonctionnement



RSA

Un exemple concret

Prenons $p = 23$ et $q = 61$, on aura $n = 23 \times 61 = 1403$. Par ailleurs :

$$\varphi(n) = \varphi(pq) = \varphi(p)\varphi(q) = (p-1)(q-1) = 22 \times 60 = 1320.$$

Il faut maintenant choisir e premier avec $\varphi(n) = 1320$. Prenons $e = 7$. Il reste maintenant à calculer d tel que $ed \equiv 1 \pmod{1320}$.

Théorème (Bezout)

Soient $a, b \in \mathbb{Z}$ non tous deux nuls. Alors, il existe $\lambda, \mu \in \mathbb{Z}$ tels que : $\lambda a + \mu b = \text{pgcd}(a, b)$. Les coefficients λ et μ peuvent être calculés de manière efficace à l'aide l'algorithme d'Euclide étendu.

Donc comme e et $\varphi(n)$ sont premiers entre eux, il existe d, f tels que : $de + f\varphi(n) = \text{pgcd}(e, \varphi(n)) = 1$, soit, modulo $\varphi(n)$, $de \equiv 1$. d est donc l'inverse de e cherché. Dans notre exemple, on peut prendre $d = 943$.

RSA

Un exemple concret

Donc, si on résume les calculs précédents, on a :

- La clé publique est la paire d'entiers $n = 1403$, $e = 7$.
- La clé privée est l'entier $d = 943$.
- Pour chiffrer le message $M = 345$, il faut calculer $M^e \pmod{n}$, ce qui nous donne $C = 345^7 \pmod{1403} = 1058$.
- Pour déchiffrer C , il faut calculer $C^d \pmod{n}$, c'est à dire évaluer $1058^{943} \pmod{1403}$. On retrouve bien $M = 345$.

Bien entendu, cet exemple n'est pas réaliste en raison de la trop petite taille de la clé publique n . La recommandation actuelle pour la taille de la clé publique n est 2048 bits.

En pratique, on fabrique rarement ses clés, on utilise plutôt des logiciels comme *OpenSSL* pour générer des clés RSA.

La notion de trappe ou trapdoor

Def: a trapdoor func. $X \rightarrow Y$ is a triple of efficient algs.
(G, F, F^{-1})

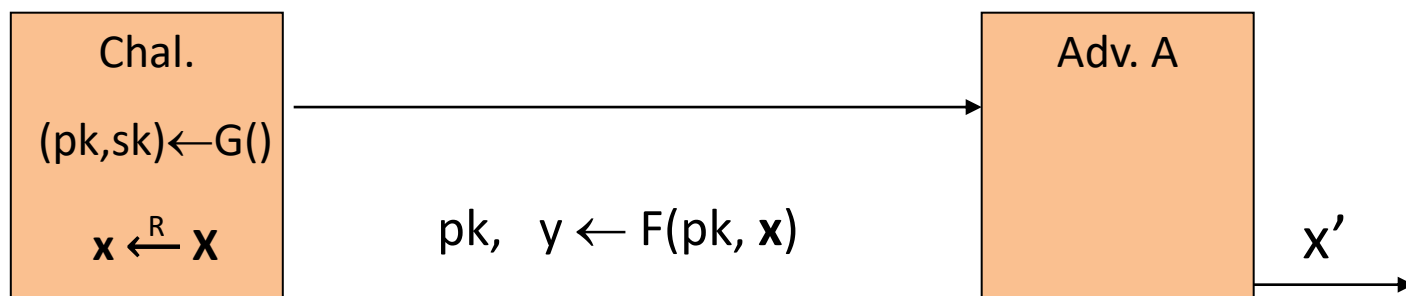
- $G()$: randomized alg. outputs a key pair (pk, sk)
- $F(pk, \cdot)$: det. alg. that defines a function $X \rightarrow Y$
- $F^{-1}(sk, \cdot)$: defines a function $Y \rightarrow X$ that inverts $F(pk, \cdot)$

More precisely: $\forall (pk, sk)$ output by G

$$\forall x \in X: F^{-1}(sk, F(pk, x)) = x$$

La notion de trappe ou trapdoor

(G, F, F^{-1}) is secure if $F(pk, \cdot)$ is a “one-way” function:
can be evaluated, but cannot be inverted without sk



Def: (G, F, F^{-1}) is a secure TDF if for all efficient A :

$$\text{Adv}_{\text{OW}}[A, F] = \Pr[x = x'] < \text{negligible}$$

La notion de trappe ou trapdoor

Protocole RSA

RSA - Chiffrement



$$\mathcal{E}(\text{pk} = (e, n), M) = M^e \pmod{n}$$

RSA - Déchiffrement



$$\mathcal{D}(\text{sk} = d, C) = C^d \pmod{n}$$

Vérification

$$(M^e)^d = M^{ed} = M^{1-u\varphi(n)} = M \cdot 1 = M \pmod{n}$$

(Théorème d'Euler)

La notion de trappe ou trapdoor

Exemple :

fonction à sens unique avec trappe secrète.

Prenons par exemple le cas de la fonction f suivante :

$f : x \rightarrow x^3 \pmod{100}$.

– Connaissant x , trouver $y = f(x)$ est facile, cela nécessite deux multiplications et deux divisions.

– Connaissant y image par f d'un élément x ($y = f(x)$), retrouver x est difficile.

Tentons de résoudre le problème suivant : trouver x tel que $x^3 \equiv 11 \pmod{100}$.

On peut pour cela :

– soit faire une recherche exhaustive, c'est-à-dire essayer successivement 1, 2, 3, ..., 99, on trouve alors :

$71^3 = 357\,911 \equiv 11 \pmod{100}$,

– soit utiliser la trappe secrète : $y \rightarrow y^7 \pmod{100}$ qui fournit directement le résultat !

$11^7 = 19\,487\,171 \equiv 71 \pmod{100}$

Trapdoor trappe RSA: est exposant d qui rend la fonction de déchiffrement facile.

La Sécurité de RSA

Problèmes difficiles



Factorisation

- $(p, q) \rightarrow p \cdot q$ facile
- $n = p \cdot q \rightarrow (p, q)$ difficile

Meilleur algorithme (crible algébrique) : $\mathcal{O} \left(e^{1.92(\ln n)^{1/3} (\ln \ln n)^{2/3}} \right)$



Fonction RSA

Extraction de racine e -ième

- $x \rightarrow x^e \bmod n$ facile
- $y = x^e \rightarrow x \bmod n$ difficile

avec la trappe $d = e^{-1} \pmod{\varphi(n)}$: $x = y^d = x^{ed} = x \bmod n$

La sécurité sémantique RSA

RSA est un algorithme déterministe et tout les algorithmes déterministes ne sont pas sémantiquement sûrs, soit le scénario suivant :

- ✓ Supposons que l'adversaire sait que l'utilisateur chiffre l'un des deux messages m_1 ou m_2 ,
- ✓ et il est supposé connaître n et e (la clé publique de l'utilisateur).
- ✓ À la réception du ciphertext c l'adversaire veut déterminer si le plaintext correspondant m est égal à m_1 ou m_2 . Tout l'adversaire a besoin de faire est calculé :

$$c' = m^e \bmod n.$$

alors

- Si $c' = c$ alors $m = m_1$,
- Si $c' \neq c$ alors $m = m_2$.

La sécurité sémantique RSA

Propriétés du chiffrement



RSA = Homomorphisme

Le chiffré d'un produit $M_1 M_2$ est égal au produit des chiffrés

$$C_1 = M_1^e \pmod{n}$$

$$C_2 = M_2^e \pmod{n}$$

$$C_1 C_2 = M_1^e M_2^e = (M_1 M_2)^e$$

Intéressant pour certains scénarios
Mais aussi nuisible à la sécurité ...



Attaque à chiffré choisi

- 1 Soit $C = M^e$ un message chiffré
- 2 On fabrique $C' = A^e M^e$ le chiffré du message $A \cdot M$
- 3 Le déchiffrement de C' fournit celui de C

La sécurité sémantique RSA

RSA est un algorithme déterministe et tout les algorithmes déterministes ne sont pas sémantiquement sûrs, soit le scénario suivant :

- ✓ Supposons que l'adversaire sait que l'utilisateur chiffre l'un des deux messages m_1 ou m_2 ,
- ✓ et il est supposé connaître n et e (la clé publique de l'utilisateur).
- ✓ À la réception du ciphertext c l'adversaire veut déterminer si le plaintext correspondant m est égal à m_1 ou m_2 . Tout l'adversaire a besoin de faire est calculé :

$$c' = m^e \bmod n.$$

alors

- Si $c' = c$ alors $m = m_1$,
- Si $c' \neq c$ alors $m = m_2$.

La sécurité sémantique RSA

Propriétés du chiffrement



RSA = Déterministe

Chiffrement déterministe = si l'on chiffre plusieurs fois le même message, on obtient le même chiffré

Méthodes pour éviter le chiffrement par substitution :

- couper le message en grands blocs
- modifier la taille de blocs à chaque fois
- randomiser le chiffrement RSA

l'algorithme RSA a été renforcé, grâce à l'utilisation de fonctions de padding avant le chiffrement. Nous appelons ce type de schéma le Padding RSA.

Rappel

■ Diffie–Hellman Key Exchange: Example

Domain parameters $p=29$, $\alpha=2$

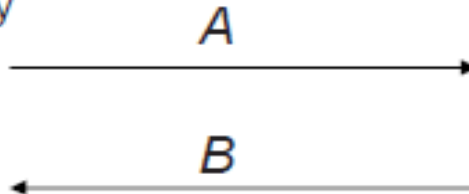
Alice

Bob

Choose random private key
 $k_{prA} = a = 5$

Choose random private key
 $k_{prB} = b = 12$

Compute corresponding public key
 $k_{pubA} = A = 2^5 = 3 \bmod 29$



Compute corresponding public key
 $k_{pubB} = B = 2^{12} = 7 \bmod 29$

Compute common secret
 $k_{AB} = B^a = 7^5 = 16 \bmod 29$

Compute common secret
 $k_{AB} = A^b = 3^{12} = 16 \bmod 29$

$$y \equiv x * K_{AB}$$

Proof of correctness:

Alice computes: $B^a = (\alpha^b)^a \bmod p$

Bob computes: $A^b = (\alpha^a)^b \bmod p$

i.e., Alice and Bob compute the same key k_{AB} !

$$x \equiv y * K_{AB}^{-1}$$

L'algorithme ElGamal

Principle of Elgamal Encryption

Alice

Bob

- (a) choose $d = k_{pr,B} \in \{2, \dots, p-2\}$
- (b) compute $\beta = k_{pub,B} \equiv \alpha^d \pmod p$

β

- (c) choose $i = k_{pr,A} \in \{2, \dots, p-2\}$
- (d) compute $k_E = k_{pub,A} \equiv \alpha^i \pmod p$

k_E

- (e) compute $k_M \equiv \beta^i \pmod p$
- (g) encrypt message $x \in \mathbb{Z}_p^*$
 $y \equiv x \cdot k_M \pmod p$

y

- (f) compute $k_M \equiv k_E^d \pmod p$

- (h) decrypt $x \equiv y \cdot k_M^{-1} \pmod p$

Protocole ElGamal

Elgamal Encryption Protocol

Alice

choose $i \in \{2, \dots, p-2\}$
compute ephemeral key
 $k_E \equiv \alpha^i \bmod p$
compute masking key
 $k_M \equiv \beta^i \bmod p$
encrypt message $x \in \mathbb{Z}_p^*$
 $y \equiv x \cdot k_M \bmod p$

$k_{pub} = (p, \alpha, \beta)$

←

(k_E, y)

→

Bob

choose large prime p
choose primitive element $\alpha \in \mathbb{Z}_p^*$
or in a subgroup of $\mathbb{Z}_p^*$
choose $k_{pr} = d \in \{2, \dots, p-2\}$
compute $k_{pub} = \beta = \alpha^d \bmod p$

compute masking key

$k_M \equiv k_E^d \bmod p$
decrypt $x \equiv y \cdot k_M^{-1} \bmod p$

Protocole ElGamal

Alice

message $x = 26$

choose $i = 5$

compute $k_E = \alpha^i \equiv 3 \pmod{29}$

compute $k_M = \beta^i \equiv 16 \pmod{29}$

encrypt $y = x \cdot k_M \equiv 10 \pmod{29}$

$\xleftarrow{k_{pub,B} = (p, \alpha, \beta)}$

$\xrightarrow{y, k_E}$

Bob

generate $p = 29$ and $\alpha = 2$

choose $k_{pr,B} = d = 12$

compute $\beta = \alpha^d \equiv 7 \pmod{29}$

compute $k_M = k_E^d \equiv 16 \pmod{29}$

decrypt

$x = y \cdot k_M^{-1} \equiv 10 \cdot 20 \equiv 26 \pmod{29}$

Protocole ElGamal

ElGamal - Chiffrement



$$\mathcal{E}(\text{pk} = y, M) = (C, D)$$

Pour un aléa r on calcule une paire (C, D) (le chiffré de M)

$$C = g^r \pmod{p}$$

$$D = M \cdot y^r \pmod{p}$$

ElGamal - Déchiffrement



$$\mathcal{D}(\text{sk} = x, (C, D)) = D \cdot C^{-x} \pmod{p}.$$

Vérification

$$D \cdot C^{-x} = M y^r (g^r)^{-x} = M (g^x)^r (g^r)^{-x} = M \pmod{p}$$

Protocole ElGamal

Emploi de ElGamal



Avantages

- Tous les utilisateurs peuvent utiliser le même groupe $\mathbb{Z}_p^*$ et le même g
- Possibilité de techniques d'exponentiation rapide
- Chiffrement randomisé nativement
- Meilleure sécurité basique



Inconvénients

- Augmentation de la taille du chiffré
- Deux fois plus lent que RSA

Protocole ElGamal

Efficacité de ElGamal



ElGamal - coût

Le coût est celui de deux exponentiations modulaires :

- El Gamal est 2 fois plus lent que RSA.
- La taille des données chiffrées représente 2 fois celle des données en clair.

Protocole ElGamal

Propriétés multiplicatives



ElGamal -Attaque à chiffré choisi

Si (C, D) est un chiffré de M , pour tout A , le couple $(C, A \cdot D)$ chiffre $A \cdot M$:

$$\begin{aligned}C &= g^r \pmod{p} \\ A \cdot D &= A \cdot M \cdot y^r \pmod{p}\end{aligned}$$

Protocole ElGamal

Attaques ElGamal



ElGamal avec le même aléa

Ne jamais utiliser deux fois le même aléa !

Si M_1 et M_2 sont chiffrés avec le même aléa r , alors :

$$(C_1, D_1) = (g^r, y^r M_1) \pmod{p}$$

$$(C_2, D_2) = (g^r, y^r M_2) \pmod{p}$$

d'où :

$$\frac{M_1}{M_2} = \frac{D_1}{D_2}$$

Protocole ElGamal



Logarithme discret (DLOG)

Définition Soit $\mathbb{G}$ un groupe multiplicatif, $g \in \mathbb{G}$ et $y \in \langle g \rangle$:

$$\log_g(y) = x \quad \text{où} \quad g^x = y$$



Réduction

La recherche de la clé privée x à partir de la clé publique $y = g^x$ est équivalente au problème du logarithme discret (DLOG).

ElGamal se réduit au logarithme discret !